SYNAP Roadmap

Roadmap

This document lists what S.Y.N.A.P. does **not** do yet, and why — not as an afterthought, but as a first-class complement to `ARCHITECTURE.md`, which describes what exists. A roadmap that only lists planned features without explaining why the current gaps exist is a marketing document; this one is meant to be useful to someone deciding whether to depend on this project, or to contribute to it.

Each item states: **what is missing**, **why it doesn't exist yet** (a scope decision, a complexity tradeoff, or a genuine unsolved problem), and, where relevant, **what it would take** to add it.

How to read this document

Three categories, not one flat list:

- **§1–§5 — Not yet implemented.** Real gaps, with a concrete reason and (where useful) a sketch of the work required. These are candidates for contribution — see `CONTRIBUTING.md`.
- **§6 — Release & distribution.** Practical, non-technical gaps (this project isn't published anywhere yet).
- **§7 — Deliberately out of scope.** Not gaps — decisions. Listed separately so they aren't mistaken for oversights.

A prioritization summary is in §8.

---

1. Statistical & Causal Modeling (Modules 1–2)

1.1 — Meek's orientation rules are not fully implemented

What's missing: Module 2's DAG-learning path (`learn_dag_from_contract`) runs a simplified PC-algorithm: skeleton phase (conditioning sets capped at size 2), v-structure orientation, and a declared-order fallback for whatever edges remain undirected. It does not apply Meek's four orientation propagation rules, which can orient additional edges purely from the graph structure already discovered (without new independence tests).

Why: implementing Meek's rules correctly requires careful handling of partially-directed graphs (a mix of directed and undirected edges) and several edge-case propagation loops. The conditioning-set cap at 2 was a deliberate, separate tradeoff to bound runtime on graphs with dozens of nodes; the two limitations compound, but were accepted together to ship a DAG-learning path that is correct on the common case (sparse causal graphs) rather than incomplete on the general case.

What it would take: a `_apply_meek_rules(dag, skeleton, separating_sets)` pass after v-structure orientation, iterating the four rules to a fixed point. Self-contained; doesn't require touching the skeleton phase.

1.2 — Categorical columns only inherit a numeric parent's influence via

an explicit DAG edge

What's missing: if a categorical column genuinely depends on a numeric one in the real data, but the user doesn't declare that edge (and the DAG is learned automatically), the dependency is lost — quantified directly in the Validation Report (§4): association recovered at 0.805 with a declared edge, ~0.0004 without one.

Why: the Contract (Module 1) only stores a Pearson correlation matrix between *numeric* columns — categorical variables have no correlation value to test for d-separation against. Module 2's automatic DAG learner therefore has no statistical signal to discover a categorical-numeric edge on its own; it can only use one if a human supplies it.

What it would take: a categorical-numeric association measure (e.g., correlation ratio / eta-squared, already used internally by the Validation Report's own analysis scripts to *check* this behavior) fed into the same conditional-independence testing framework used for numeric-numeric pairs.

1.3 — No time-series / sequential data support

What's missing: every column is treated as an independent observation row; there is no notion of a time index, autocorrelation, or sequence-level generation (e.g., "generate a plausible 12-month transaction history per customer").

Why: out of scope for the initial design, which targets flat, cross-sectional tabular data (the regulated-sector use cases in `PROJECT_NARRATIVE.md` — credit risk, insurance, healthcare records — are predominantly of this shape). Adding sequence support is a different modeling problem (state-space or autoregressive generation) layered on top of, not instead of, the current per-row architecture.

1.4 — No relational / multi-table schemas

What's missing: one Contract, one DAG, one output table. There is no way to declare two related tables (e.g., `customers` and `transactions`) and generate both while preserving referential integrity and cross-table statistics — a capability that, notably, the open-source **SDV** project does support and is one of its main differentiators from this project (see the comparison table in `PROJECT_NARRATIVE.md`, §VI).

Why: every module from the Contract onward (correlation matrix, DAG, Mahalanobis filter, Twin Model) assumes a single flat table. Multi-table support would require a schema layer above the current one (table definitions + foreign-key relationships) and would touch most of the 8 modules, not just Module 1 — a substantial redesign, not an incremental addition.

---

2. Logical Validation & Rule Induction (Module 4)

2.1 — The ILP engine is not general-purpose

What's missing: rule induction is restricted to one template — `IF numeric_column > threshold THEN categorical_column = value` — searched over a small quantile grid (`ILP_CANDIDATE_QUANTILES`). It cannot induce rules with multiple conditions (conjunctions), disjunctive rules, rules over multiple categorical columns, or rules with a numeric consequent.

Why: a genuinely general ILP engine (multi-variable hypothesis search, e.g. FOIL- or Aleph-style) is a research-grade undertaking on its own; the univariate template covers the example given in the

module's own specification (`income > 5000 → risk = low`) and keeps the search space — and therefore the per-checkpoint computational cost — bounded and predictable.

What it would take: extending `induce_rules` to search conjunctions of 2+ numeric conditions is the smallest useful next step; true disjunctive/multi-column rules would need a different search strategy (e.g., greedy set-covering) rather than a grid search.

2.2 — No user-defined custom validation plugins

What's missing: business rules are limited to the `"column operator value"` string grammar (`>` , `>=` , `<` , `<=` , `==` , `!=`). There is no way to register an arbitrary Python function as a custom row-level or dataset-level validator.

Why: the current grammar is deliberately simple so that a rule is also, unambiguously, a hard logical constraint that the ASP layer can enforce with zero exceptions (see `ARCHITECTURE.md` §6). An arbitrary Python callable can't be reasoned about the same way (it can't be tested for consistency against the DAG, for instance) without additional constraints on what it's allowed to do.

What it would take: a `CustomRule` protocol (a callable taking a row and returning bool) that plugs into `validate_row` alongside `BusinessRule` , with an explicit note that such rules are opaque to Module 4's causal-consistency checks.

---

3. Privacy (Module 8)

3.1 — No formal differential privacy guarantee

What's missing: Module 8's privacy checks (DCR, NNDR, exact-match, MIA) are **empirical audits** — they measure whether memorization or membership leakage is *observed* in a specific generated dataset. They are not a **formal** differential-privacy (DP) mechanism, which would provide a mathematical guarantee (an ε, δ budget) that holds regardless of what a future attacker might try, including attacks that don't resemble any of the ones tested.

Why: formal DP requires calibrated noise injection *inside* the generative process itself (e.g., DP-SGD during model training, or noise-calibrated statistics in Module 1's Contract construction) — architecturally different from an audit-then-veto design. Retrofitting DP onto an already-built pipeline is possible in principle but would touch the Contract, the seed generation, and the homeostasis correction loop simultaneously, since each of those currently uses exact statistics.

What it would take: the most tractable path is a differentially private Contract (Module 1) — adding calibrated noise to the mean/std/ correlation estimates computed from the real anchor — which would give the *statistics that everything downstream is built from* a formal guarantee, without redesigning Modules 2–7. This is the natural next step, not a full rewrite, but it is a real, non-trivial project on its own.

3.2 — The Membership Inference Attack is intentionally simple

What's missing: Module 8's MIA (`membership_inference_risk`) is a single-feature, distance-based attacker (nearest-synthetic-neighbor distance, fed into a logistic regression). It is not a shadow-model attack (training multiple "shadow" generators to model the target generator's behavior more closely), which is generally considered a stronger, more realistic adversary in the academic membership-inference literature.

Why: shadow-model attacks require training multiple additional generative models per audit — for

a system whose own generation is already the expensive part of the pipeline, this multiplies the cost of every privacy audit by the shadow-model count. The distance-based attacker is a legitimate, established technique (and the Validation Report shows it produces a sensible, calibrated result: AUC ≈ 0.5 on safe data, materially above 0.5 on deliberately memorized data), but it is a weaker adversary than the state of the art.

What it would take: an alternative `membership_inference_risk` implementation using `k` shadow models trained on resampled subsets of the anchor data, exposed as an opt-in (more expensive) alternative to the current default.

---

4. Scale & Performance

4.1 — Single-process, single-machine execution

What's missing: there is no distributed or multi-process execution path. `synap.main.run_synap` runs the entire loop — generation, validation, homeostasis — in one Python process. The Flask web server (`synap.webserver`) runs generation jobs in background *threads*, which provides responsiveness (the HTTP request doesn't block), not parallelism (Python's GIL means CPU-bound generation work in one job doesn't speed up by adding more threads, and two simultaneous jobs contend for the same CPU).

Why: the reservoir-capped memory design (Module 3/5, `MAX_MEMORY_SIZE`) and the incremental sufficient-statistics accumulators (Modules 4 and 6) were specifically built to bound *memory* growth to support large row counts on one machine — that problem was solved deliberately. Distributing *computation* across processes or machines is a different problem (partitioning the chimera-remix memory pool, or running independent batches on different workers and merging homeostasis state) that hasn't been tackled.

What it would take: the most contained starting point is multi-process batch generation (`multiprocessing` or a task queue like Celery/RQ) with a shared, periodically-synchronized `HomeostaseState` — full distribution across machines would additionally need the `RunningCovarianceAccumulator` and `HipotalamoState` to support merge operations, which they don't today.

4.2 — No chunked/streaming output for very large datasets

What's missing: the generated dataset accumulates as a single in-memory `pandas.DataFrame` (`SynapResult` holds it as one object) until the entire run completes, then is written once. There is no incremental write-to-disk (e.g., append-mode Parquet/CSV per batch) for datasets whose final size wouldn't comfortably fit in memory.

Why: every batch already passes through Module 7's final validation and Module 8's audit *as a whole dataset*, not incrementally — the Twin Model and the DCR/NNDR calculations are computed once, over the complete result. Streaming the *output* file is straightforward; streaming the *validation*, so that Modules 7–8 don't need the full dataset in memory either, is the part that hasn't been designed.

---

5. Web Interface & Deployment (`synap.webserver` , `site/`)

5.1 — Job state is in-memory only

What's missing: `synap.webserver.JOBS` is a plain Python dictionary. Restarting the `synap --serve-site` process loses all job history and in-progress jobs; there's no persistence (SQLite, Redis, or otherwise) and no way to list past runs.

Why: the server was built for a single local user running the tool interactively, where a restart is a rare, deliberate action, not a production multi-user deployment.

5.2 — No authentication or multi-tenancy

What's missing: any client that can reach the server's port can call `/api/generate` and read any `job_id`'s results — there is no login, no per-user job isolation, no rate limiting.

Why: `synap --serve-site` binds to `127.0.0.1` (loopback only) by design — it is explicitly a local development/demonstration tool, not intended to be exposed on a shared network or the public internet as-is.

What it would take: if this became a shared/hosted tool, it would need, at minimum: a proper WSGI/ASGI production server (not Flask's built-in development server), authentication, per-user job namespacing, and request rate limiting — a genuinely different deployment target from today's "run it on your own laptop" model.

5.3 — The website's data-generation form doesn't work from a static file

What's missing: opening `site/index.html` directly (`file://`, or a static host like GitHub Pages) renders the presentation and animations, but the "Generate Data" section has no backend to call — it requires `synap --serve-site` running locally.

Why: this is the direct, intended consequence of connecting the website to the *real* Python pipeline (as opposed to the earlier, removed `webapp/synap_webapp.html`, which was a JavaScript reimplementation that worked with no server at all — see §7.1). A fully static "real pipeline in the browser" would require compiling the Python stack (NumPy, scikit-learn, SciPy) to WebAssembly via Pyodide, which is possible in principle but hasn't been attempted.

---

6. Release & Distribution

Not technical gaps — things simply not done yet:

- **Not published to PyPI.** `pip install -e .` (editable, from a local clone) works and is tested in CI; there is no published package to `pip install synap` from.
- **No GitHub repository exists yet.** URLs like `github.com/synap-project/synap` throughout the docs, `pyproject.toml`, and `CITATION.cff` are placeholders, flagged as such wherever they appear.
- **No auto-generated API reference site** (e.g., a Sphinx or MkDocs build of the docstrings). `ARCHITECTURE.md` documents the modules at a design level; there's no browsable per-function reference beyond reading the source.
- **No enforced coverage threshold in CI.** `pytest --cov` runs and reports, but CI does not currently fail a PR for a coverage regression.

---

7. Deliberately out of scope

Listed separately from §1–§6 because these are not gaps waiting to be filled — they're decisions, made and reconsidered, kept out on purpose.

7.1 — The standalone JavaScript reimplementation of the pipeline

An earlier version of the website (`webapp/synap_webapp.html`) reimplemented simplified equivalents of all 8 modules in vanilla JavaScript — genuinely functional, running entirely client-side, no server required. It was **deliberately removed** once the website was connected to the real Python backend (`synap.webserver`), because maintaining two parallel implementations of the same system (one real, one a JavaScript approximation) invites exactly the kind of confusion this roadmap is trying to avoid: which one is "the system"? The real pipeline, reachable via `synap --serve-site` , is the only one now.

7.2 — Non-tabular data modalities

Images, free text, audio, and other unstructured data types are outside this project's problem space. Every module — from the Contract's mean/std/correlation model to the Mahalanobis filter to the Twin Model — assumes structured, tabular columns. Supporting unstructured modalities would not be an extension of this architecture; it would be a different project.

7.3 — Bilingual code and error messages

Function names, docstrings, and log/warning messages throughout the Python codebase are in European Portuguese, by deliberate convention (stated in `CONTRIBUTING.md`), while `ARCHITECTURE.md` , `ROADMAP.md` , and `CITATION.cff` are in English for presentation purposes. This is not an inconsistency to fix — it reflects where the code was written versus where the project is presented.

---

8. Prioritization summary

Item	Category	Status
Meek's rules (§1.1)	Causal modeling	Under consideration
Categorical-numeric auto-DAG (§1.2)	Causal modeling	Under consideration
Differentially private Contract (§3.1)	Privacy	Likely next
Conjunctive ILP rules (§2.1)	Rule induction	Under consideration
Shadow-model MIA (§3.2)	Privacy	Under consideration
Multi-process batch generation (§4.1)	Performance	Under consideration
Time-series support (§1.3)	Modeling scope	Not planned (would need a dedicated design)
Relational/multi-table schemas (§1.4)	Modeling scope	Not planned (would need a dedicated design)
Custom validation plugins (§2.2)	Extensibility	Under consideration
Streaming output (§4.2)	Performance	Not planned until §4.1 is addressed
Job persistence (§5.1)	Web interface	Not planned (local-tool scope)

Auth / multi-tenancy (§5.2)	Web interface	Not planned (local-tool scope)
WASM/Pyodide static generation (§5.3)	Web interface	Not planned
PyPI publication (§6)	Distribution	Likely next
Auto-generated API docs (§6)	Distribution	Likely next

“Likely next” reflects items that are self-contained and don’t require redesigning multiple modules at once. “Under consideration” reflects real value but a larger or less-certain scope. “Not planned” reflects either an explicit non-goal (§7) or a dependency on a bigger unresolved question (e.g., streaming output depends on solving distributed execution first).